

PORTFOLIO · BACKEND DEVELOPER

박성현

Backend Developer

기능 구현에서 끝나지 않고, 운영 환경과 데이터 흐름·성능 병목까지 확인하는 Java/Spring 백엔드 개발자

Target	신입 Java / Spring 백엔드 개발자
--------	--------------------------

Email	pshyun129@gmail.com
-------	---------------------

GitHub	https://github.com/pshyun129
--------	---

Education	수원대학교 컴퓨터SW전공 · SSAFY 15기
-----------	---------------------------



박성현

BACKEND DEVELOPER

기능 구현을 넘어, 운영 흐름과 성능까지
확인하는 백엔드 개발자입니다.

PROFILE

Birth	2001.08.26
Education	수원대학교 컴퓨터SW전공 · SSAFY 15기
Active	알고리즘 문제 풀이 스터디 운영 2025.06 — 2025.12
	교수 지도 헬스케어 서비스 개발 프로젝트 2024.03 — 2024.07

AWARDS

2025 한이음 드림업 공모전
동상 · 2025.12

2024 한이음 ICT 공모전
장려상 · 2024.12

PROJECTS

Ait	2026.07 — 2026.08
AI 맞춤형 모의면접 및 실시간 화상 면접 스터디 플랫폼 문서·AI 서버 연동, LiveKit 실시간 통신, 배포 환경 구성과 성능 테스트를 담당했습니다.	
BRIX	2025.03 — 2025.11
AI 기반 과일 당도 예측·자동 분류 및 직거래 플랫폼 Spring Security·JWT 인증, Redis 기반 토큰·인증번호 관리, S3 이미지 생명주기와 인기 상품 캐싱을 구현했습니다.	
Everyday Friends	2024.03 — 2024.11
라즈베리파이 기반 감정 분석 맞춤형 힐링 서비스 Spring Boot·MySQL 기반 백엔드와 세션 인증, 감정 상태 기반 콘텐츠· 광고 REST API를 구현했습니다.	

Tech Skills

BACKEND

Java Spring Boot JPA
Spring Security MyBatis
Gradle Maven REST API

DATABASE / CACHE

MySQL AWS RDS Redis

Refresh Token, 인증번호 TTL, 조회 캐싱에
활용

INFRASTRUCTURE

AWS EC2 AWS S3 Docker
Docker Compose Nginx
HTTPS/TLS Linux

REAL-TIME / NETWORK

LiveKit WebRTC WebSocket
LiveKit Webhook

Room/Token/Webhook 연동 및 Docker 서
버 운영

Ait

AI 맞춤형 모의면접과 실시간 화상 면접 스터디를 통합한 개발자 면접 준비 플랫폼

Team	AI 1 · Frontend 2 · Backend 3
My Role	Backend · Infrastructure · Performance Testing
Core	Spring Boot · Redis · LiveKit · Nginx · EC2 · k6
GitHub	github.com/pshyun129/AIT

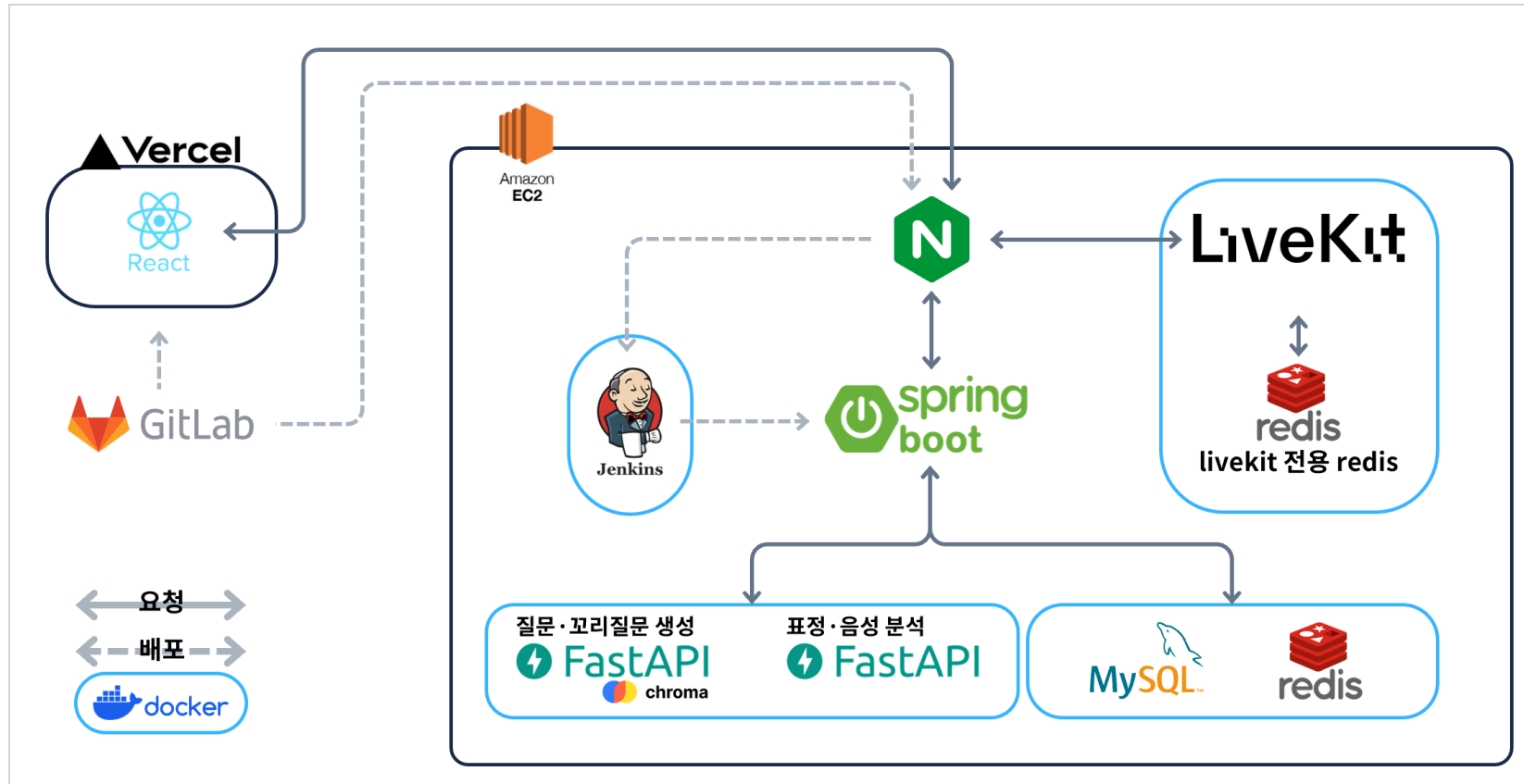
OVERVIEW

사용자가 등록한 이력서·자기소개서·Repository 정보를 바탕으로 개인화된 면접 질문을 생성하고, 답변·표정·음성 분석 결과를 종합해 리포트를 제공한다. LiveKit 기반 화상 스터디와 참여자 상호평가도 함께 제공한다.

MY ROLE

- Spring Boot 백엔드 API 개발
 - 이력서·자기소개서 CRUD
 - GMS 문서 분석 연동
 - 이벤트 기반 비동기 분석
 - NFC 정규화·편집 거리 재분석 조건
 - Spring Boot ↔ FastAPI 연동
 - LiveKit 서버 구축·연동
 - Room·Token·Webhook 처리
- 스터디 세션·상호평가 API
 - EC2 배포
 - Docker LiveKit·Redis 운영
 - Nginx Reverse Proxy-HTTPS
 - 이미지 업로드·조회·다운로드 경로
 - k6 사용자 시나리오 부하 테스트
 - Linux 시스템 지표 병목 분석
 - Redis 캐싱·재측정

ARCHITECTURE



Nginx가 HTTPS 진입점을 맡고 Spring Boot가 사용자·서비스 상태를 관리하며, Redis·MySQL과 외부 AI/실시간 시스템을 연결한다.

KEY
IMPLEMENTATIONS

01	문서 저장과 AI 분석 책임 분리	이력서·자기소개서 저장 요청과 GMS 분석 작업을 이벤트 기반 비동기 구조로 분리해, 사용자의 저장 흐름과 처리 시간이 긴 분석 작업을 독립적으로 처리했다.
02	변경률 기반 재분석 정책	문서 수정 시 NFC 정규화 후 Levenshtein Distance로 변경 비율을 계산하고, 10% 이상 변경된 경우에만 재분석 이벤트를 발생시켰다.
03	LiveKit 실시간 환경 구축	사용자·세션 검증 후 개별 Access Token을 발급하고, 최대 8명 Room 제한·Docker 운영·Redis 연동·WebRTC용 TCP/UDP 포트를 구성했다.
04	서비스 상태와 실시간 Room 상태 연결	LiveKit Room과 별개로 스터디 세션 참여 상태를 관리하고, Webhook 이벤트를 Spring Boot Endpoint에 연결해 실시간 상태와 서비스 데이터를 연계했다.
05	평가 데이터 완결성 검사	방장이 세션을 종료하기 전, 각 참여자가 본인을 제외한 모든 참여자에게 상호평가를 제출했는지 검사하고 미제출 사용자 정보를 반환하는 API를 만들었다.

PERFORMANCE

단일 API 반복 호출 대신 실제 사용 순서인 로그인 → 대시보드 → 내 스터디 그룹 → 그룹 상세 → 면접 리포트를 시나리오로 구성했다.
k6 지표와 CPU·Run Queue·Java Process 같은 시스템 지표를 동시에 수집해 병목을 판단했다.

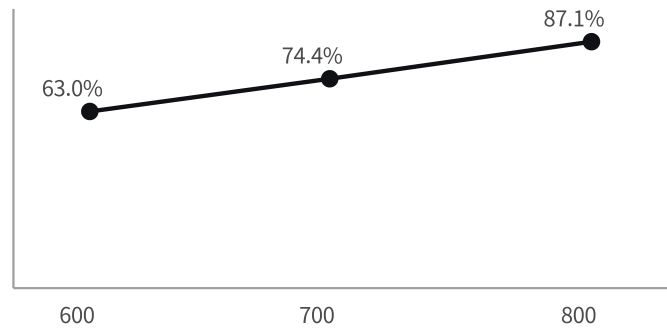
Redis 캐싱 적용 전 부하 테스트 결과

부하	평균 RPS	평균 응답	1초 단위 p95 중앙값	CPU 평균	Run Queue 평균
200 VU	93.2	30.7 ms	33.4 ms	29.2%	1.66
300 VU	138.8	35.0 ms	44.5 ms	41.7%	2.61
400 VU	185.1	35.0 ms	51.0 ms	55.4%	3.99
500 VU	230.5	44.9 ms	68.0 ms	70.0%	6.21
600 VU	272.3	78.0 ms	130.8 ms	87.8%	11.19

Redis 캐싱 적용 후 부하 확대 결과

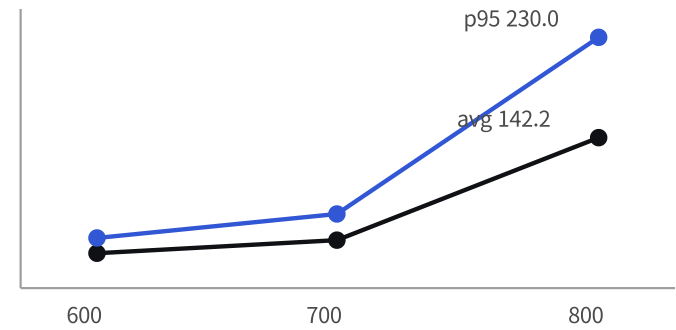
부하	평균 RPS	평균 응답	p95	CPU 평균	Run Queue
600 VU	278.4	28.3 ms	42.2 ms	63.0%	4.94
700 VU	323.0	39.1 ms	63.7 ms	74.4%	6.99
800 VU	352.7	142.2 ms	230.0 ms	87.1%	11.91

CPU 평균 / VU



700 VU까지는 완만한 증가, 800 VU에서 CPU가 다시 높은 수준으로 상승했다.

응답시간 / VU



800 VU에서 평균 142.2ms, p95 230ms까지 증가해 요청 처리 대기가 커진 구간을 확인했다.

Redis 캐싱 적용 후 동일 조건으로 다시 검증하고, 더 높은 부하에서 새 한계를 찾았다.

성능 개선 결과

87.8% → 63.0%

600 VU CPU 사용률 · 캐싱 적용 전후

24.8%p

CPU 사용률 감소 폭

800 VU

응답시간·CPU·Run Queue가 다시 급증한 새 포화 구간

Problem	사용자 행동 기반 테스트에서 600 VU 구간의 서버 CPU 사용률이 약 87.8%까지 상승했다.
Analysis	대시보드·스터디·리포트 등 반복 조회 비중이 높은 흐름을 확인하고, 변경 빈도가 낮은 데이터가 요청마다 조회 로직과 DB 접근을 반복한다고 판단했다.
Decision	조회 경로 전체를 변경하기보다 반복 조회 데이터를 Redis 캐시에 두고 동일 데이터 요청의 DB 접근과 기존 조회 로직을 줄이는 방향을 선택했다.
Implementation	주요 조회 데이터에 캐싱을 적용한 뒤 동일한 600 VU로 재테스트했다. 개선 후 700~800 VU까지 부하를 확장했다.
Result	600 VU CPU 사용률 약 87.8% → 63.0% 수준, 24.8%p 감소. 800 VU 부근에서 평균 응답 142.2ms·p95 230ms·CPU 평균 87.1%·Run Queue 11.91로 다시 포화 징후를 확인했다.
What I learned	성능 개선은 한 번의 수치 개선으로 끝내지 않고, 동일 조건 재검증과 부하 확대를 통해 새로운 병목을 다시 찾는 과정까지 포함해야 한다.

작은 문서 수정까지 전체 AI 분석을 발생시키던 구조를 조건부 재분석 방식으로 바꿨다.

Problem	오타자·띄어쓰기 같은 작은 수정에서도 전체 문서를 GMS에 다시 보내 LLM 토큰이 반복 소비됐다.
Analysis	문서를 여러 번 다듬는 실제 사용 패턴에서는 수정 발생 = 항상 재분석 구조가 호출 횟수와 비용을 누적시킨다고 판단했다.
Decision	문자열 표현 차이로 인한 오판을 줄이기 위해 먼저 Unicode NFC 정규화를 적용하고, Levenshtein Distance로 변경 비율을 계산하기로 했다.
Implementation	변경 비율 10% 이상일 때만 재분석 이벤트를 발생시키고, 분석 작업은 문서 저장 요청과 분리해 이벤트 기반 비동기로 처리했다.
Result	작은 수정마다 전체 분석을 반복하던 구조를 의미 있는 수준의 변경이 있을 때만 재분석하는 구조로 변경했다. 정확한 토큰 절감률은 측정하지 않았다.
What I learned	AI API 연동에서도 기능 호출 자체보다 비용과 호출 빈도를 애플리케이션 레벨 정책으로 제어하는 것이 중요하다.

Webhook Timeout을 전체 시스템 문제가 아닌 네트워크 구간 문제로 나눠 범위를 좁혔다.

Problem	Docker Container의 LiveKit이 Host의 Spring Boot Webhook Endpoint로 요청할 때 <code>host.docker.internal</code> 연결이 Timeout됐다.
Analysis	LiveKit Container → Docker Host → Nginx → Spring Boot로 경로를 나누고 각 구간의 접근 가능 여부와 Endpoint 도달 여부를 확인했다.
Decision	Docker 내부 주소에 직접 의존하지 않고 외부 HTTPS Domain과 Nginx Reverse Proxy를 통하는 경로로 재구성했다.
Implementation	Webhook 목적지를 외부 HTTPS 경로로 변경하고 Spring Boot Endpoint의 HTTP 응답을 단계적으로 확인했다.
Result	LiveKit 이벤트가 Nginx를 거쳐 Spring Boot까지 전달되는 경로를 구축했다.
What I learned	복합 네트워크 문제는 한 번에 설정을 바꾸기보다 요청 경로를 구간별로 검증하면 원인 범위를 더 빠르게 좁힐 수 있다.

세션 종료와 평가 데이터의 완결성을 연결했다.

Problem	방장이 먼저 나가면 세션이 종료되어, 아직 평가를 제출하지 않은 참여자가 상호평가를 작성할 수 없었다.
Analysis	세션 상태 변화와 평가 완료 여부가 서로 독립적으로 처리되어 사용자 조작 순서에 따라 데이터가 누락될 수 있었다.
Decision	세션 종료 전 각 참여자가 본인을 제외한 모든 참여자에게 평가를 작성했는지 서비스 레벨에서 검사하도록 했다.
Implementation	전체 완료 시 <code>true</code> , 미완료 시 <code>false</code> 와 미제출 사용자 정보를 반환하는 API를 구현했다.
Result	프론트가 종료 전에 미작성 참여자를 확인할 수 있어 조기 종료로 인한 평가 데이터 누락을 방지할 수 있게 했다.
What I learned	CRUD 외에도 서비스 상태 전환이 데이터 완결성에 미치는 영향을 비즈니스 로직으로 연결해야 한다.

BRIX

RGB 카메라와 AI 기반 과일 당도 예측·자동 분류 장치, 모바일 직거래 플랫폼을 연결한 시스템

Type	2025 한이음 드림업 프로젝트
Team	AI 2 · Backend 2 · Frontend 1 · Mentor 1
My Role	Backend Developer
GitHub	github.com/pshyun129/25_HC122

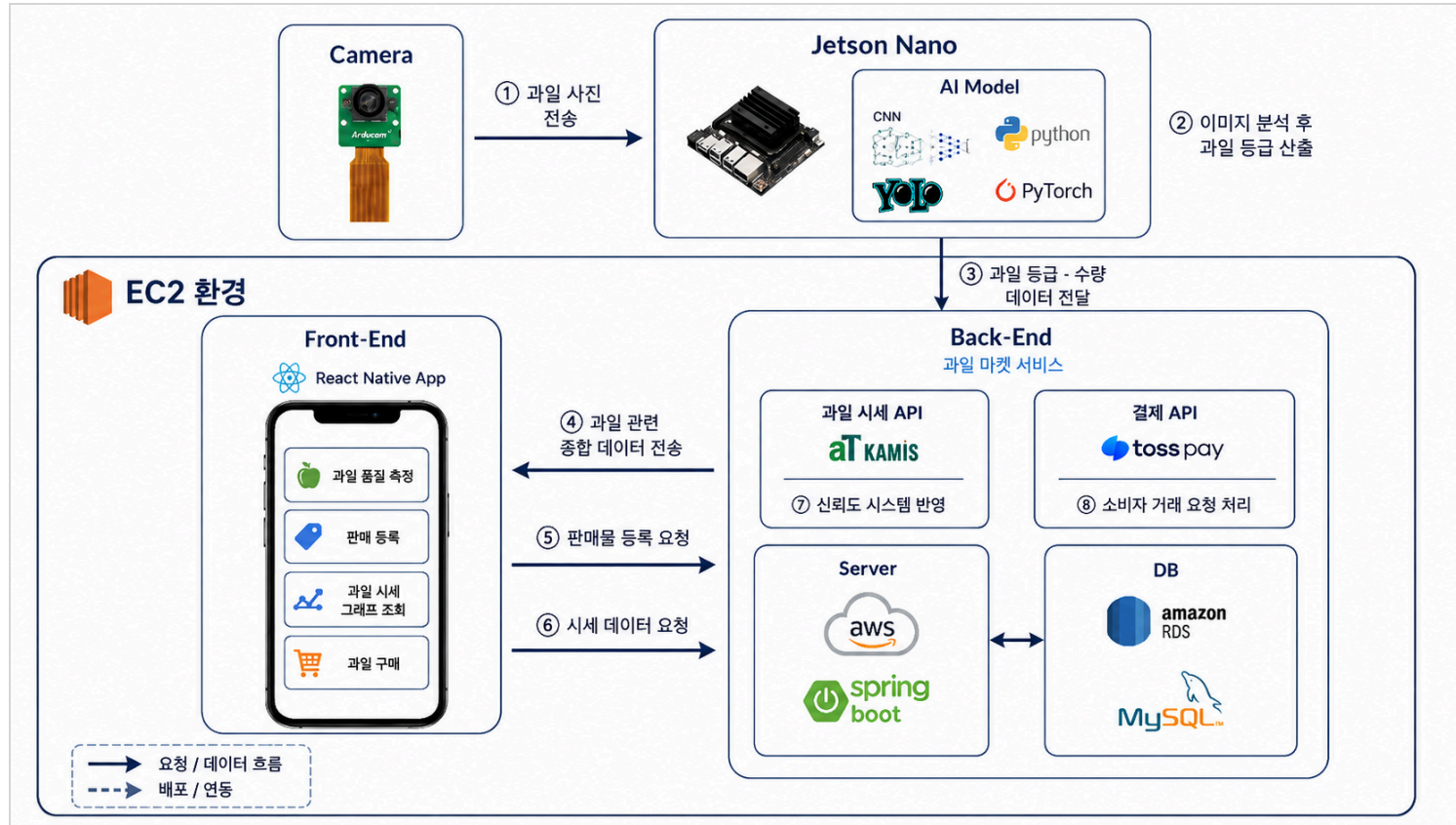
OVERVIEW

RGB 카메라와 AI 모델(CNN + YOLO)을 활용하여 과일의 당도를 비파괴적으로 측정하고, Jetson Nano 기반 자동 분류 장치 및 모바일 직거래 플랫폼과 연동하는 스마트 농업 서비스

MY ROLE

- 사용자 회원가입·로그인·회원 관리
- JWT 기반 인증·인가
- Redis 기반 Refresh Token 관리
- 이메일 인증 및 비밀번호 재설정
- AWS S3 기반 프로필·상품 이미지 관리
- 리뷰 등록·조회·삭제
- 인기 상품 조회 및 Redis 캐싱
- 과일 등급 검사 결과 관리
- Jetson Nano 메인 컨트롤 코드 구현

ARCHITECTURE



- 카메라로 촬영한 과일 이미지를 Jetson Nano의 AI 모델이 분석하여 과일 등급을 판별하고, 결과 데이터를 Spring Boot 서버로 전달한다.
- 백엔드는 과일 정보와 판매 데이터를 관리하며 KAMIS 시세 API와 결제 API를 연동해 시세 조회 및 거래 기능을 제공한다.
- React Native 앱에서는 과일 품질 측정, 판매 등록, 시세 조회, 과일 구매 등의 기능을 사용할 수 있으며, 주요 데이터는 AWS EC2와 RDS 환경에서 관리된다.

KEY
IMPLEMENTATIONS

01	JWT + Redis 인증 구조	Access/Refresh Token을 분리하고 Refresh Token을 Redis에 저장해 재발급 검증과 로그아웃 시 무효화를 처리했다.
02	Redis TTL 기반 이메일 인증	인증번호를 애플리케이션 메모리에서 분리해 Redis에 저장하고 TTL로 자동 만료되도록 구성했다.
03	S3 이미지 생명주기 관리	profiles/, products/ 경로와 UUID 파일명을 사용하고 데이터 수정·삭제 시 기존 객체도 함께 제거했다.
04	인기 상품 캐싱	변경 주기가 짧지 않은 반복 조회 데이터를 Redis에 저장하고, 캐시 미스 시 DB 조회 후 재저장하는 fallback 구조를 적용했다.
05	과일 등급 결과 서비스 연계	판매자·과일 종류 기준 S/A/B/C 등급 결과를 누적 관리해 AI·하드웨어 결과를 서비스 데이터로 연결했다.

Additional system integration — Jetson Nano 자동 분류 장치의 메인 컨트롤 코드를 직접 전체 구현했다.

이메일 인증 상태를 애플리케이션 메모리에서 Redis로 분리했다.

Problem	서버 재시작 시 인증번호가 사라지고, 여러 인스턴스로 확장될 경우 인증 상태 공유가 어려운 구조였다.
Analysis	인증번호는 애플리케이션 인스턴스의 생명주기와 분리되고 일정 시간 후 자동 만료되어야 하는 임시 상태였다.
Decision	공유 저장소와 TTL을 함께 제공하는 Redis로 인증 상태를 옮겼다.
Implementation	이메일 기반 Key로 인증번호를 저장하고 TTL을 적용했다.
Result	인증 상태가 특정 애플리케이션 인스턴스의 메모리에 의존하지 않는 구조로 변경되었다.
What I learned	상태의 수명과 공유 범위를 먼저 정의하면 저장 위치 선택의 기준이 명확해진다.

반복 조회되는 인기 상품을 요청마다 다시 계산하지 않도록 캐싱했다.

Problem	여러 사용자가 동일한 인기 상품 데이터를 반복 조회하는데 요청마다 DB 조회·계산이 반복될 수 있었다.
Analysis	인기 상품은 일반 실시간 데이터보다 변경 주기가 짧지 않아 모든 요청에서 동일 연산을 수행할 필요가 없다고 판단했다.
Decision	일정 주기로 캐시를 갱신하고 일반 조회에서는 Redis 결과를 우선 사용하도록 했다.
Implementation	캐시 미스 시 DB 조회 → Redis 저장 → 결과 반환 fallback을 구성했다.
Result	반복 조회 경로와 DB 조회 로직을 분리한 캐시 구조를 직접 설계·적용했다.
What I learned	캐시는 무조건 적용하기보다 데이터 변경 빈도와 반복 조회 패턴을 기준으로 선택해야 한다.